

CS 527 Research Project Report: Group-7

Semantic-Aware Regression Test Selection Using Large Language Models

Ankit Chavan, Saurabh Darekar
University of Illinois at Urbana-Champaign
[auc3,sdare1]@illinois.edu

Abstract

Modern software systems rely on continuous integration pipelines where regression testing is crucial for catching new bugs. However, re-running the entire test suite after every code change (“retest-all”) is increasingly impractical as it consumes excessive compute resources and delays developer feedback[1][3]. Traditional regression test selection (RTS) techniques reduce this cost by choosing only tests “impacted” by a change, using either static dependency analysis or dynamic execution tracing. Yet static methods tend to be overly conservative while dynamic methods incur heavy instrumentation and runtime overhead[2].

We propose a semantic-aware, LLM-driven RTS pipeline that analyzes the intent of code changes and test descriptions. By embedding code diffs and test summaries in a vector database (a retrieval-augmented generation approach) and incorporating test criticality tiers, our technique bridges the semantic gap between code edits and relevant tests[4]. This should yield a smaller yet safe test subset, reducing total testing time without missing important failures. We will validate the approach on the Defects4J benchmark, measuring both failure recall and test suite reduction against baseline RTS methods.

1 Introduction

Efficient regression testing is critical as software scales. Without intelligent selection, developers must often wait for the entire test suite to run after each change, slowing down development. In fact, running full test suites can take hours or even days for large projects[3]. Such delays not only reduce developer productivity but also increase infrastructure costs. Therefore, smart test selection that runs fewer tests while still catching regressions is a vital goal.

Existing RTS techniques tackle this problem in different ways. Static RTS methods analyze code dependencies at compile time to find tests that might be affected by a change. For example, STARTS builds a dependency graph of classes and selects any test whose transitive closure includes a changed type[1]. Similarly, Leung et al. performed class-level “firewall” analysis to identify dependent modules. Recent studies show that class-level static selection can indeed cut testing costs significantly[1][3]. However, static methods can be too conservative: even a minor refactoring or variable rename may cause all tests that transitively depend on the changed code to be re-run, even if no behavior actually changes[2].

Dynamic RTS methods instead collect test-to-code dependencies at runtime. Tools like Ekstazi instrument test executions to record which files or methods each test covers, and then only re-run tests whose recorded coverage overlaps the changed code. In practice, dynamic selection often executes a much smaller subset of tests. For instance, in one evaluation Ekstazi reduced end-to-end testing time

to ~53–68% of running all tests[5]. The trade-off is that dynamic approaches incur instrumentation overhead during every build. In real-time or large-scale systems, this overhead can be non-trivial and sometimes unacceptable[3].

More recently, machine learning has entered the picture. Meta’s predictive test selection, for example, trains a model on historical commits and test outcomes to predict which tests are likely to fail for a new change. In practice, this ML system catches ~99.9% of regressions while running only ~33% of tests[2]. Such results demonstrate that learned semantic patterns can be very effective. However, training ML models typically requires large amounts of data and a fixed feature space.

In summary, no existing technique perfectly balances precision (running only necessary tests) and safety (catching all failures). Static analysis errs on the side of safety at the cost of running extra tests, while dynamic analysis reduces work but adds run-time overhead[3]. Both approaches also struggle when code uses reflection or third-party libraries, since implicit dependencies may be missed or over-reported. Our semantic-aware LLM-based approach aims to bridge this gap. Instead of relying solely on code reachability, we will interpret the intent of changes and tests. An LLM can understand, for example, that changing a “Privacy Settings” feature should trigger certain security tests but not unrelated UI tests. We will encode these semantics by summarizing each test’s purpose and using an LLM to compare change intent with test intent[6]. Additionally, we assign a safety tier to each test (e.g. critical security tests) so that even if a critical test’s semantic score is low, it is still run (the “safety bridge”). In this way, the pipeline selects a concise test set that is semantically relevant to the change yet still safe.

Our experimental plan is to evaluate on the Defects4J dataset(v2.0), a standard suite of real Java bugs with associated failing tests[7]. We will preprocess all tests to build a vector database of LLM embeddings (Phase 1). Then for each buggy commit in Defects4J we will use an LLM to analyze the diff, retrieve top-K similar test embeddings (Phase 2), and apply our ranking logic (Phase 3). We will compare our method against the Retest-All baseline and a static RTS tool (e.g. STARTS) using metrics like recall of failing tests and percentage of tests run. We expect to find that our technique significantly reduces the executed test suite while still catching essentially all regressions.

2 Proposed Technique

2.1 Approach Overview

Our approach is a multi-stage agentic pipeline that uses an LLM at its core. The key idea is to treat each test as a semantically-described object and each code change as a semantic query, then

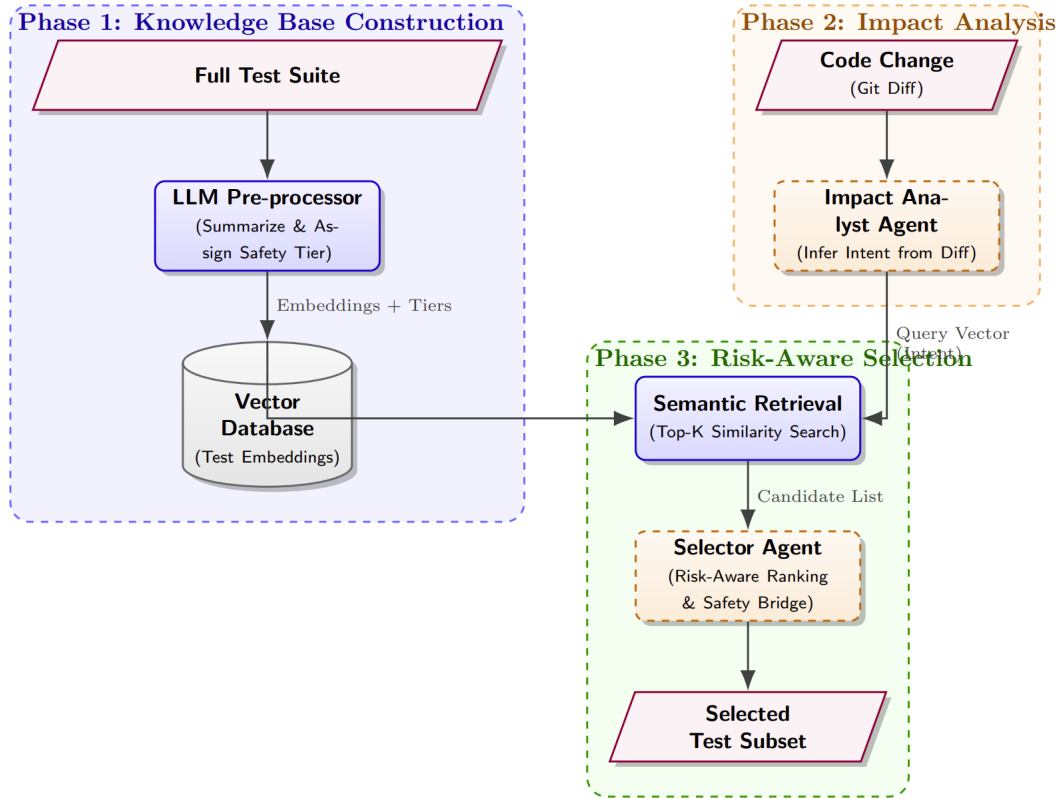


Figure 1: Overview of the Semantic-Aware RTS Framework. The pipeline consists of (1) offline vectorization of tests and safety tiers, (2) runtime intent analysis of code changes, and (3) risk-aware ranking using a “Safety Bridge” to prioritize critical tests.

match them via an embedding-based retrieval. The pipeline has three main phases:

- **Knowledge Base Construction** : We preprocess the entire test suite using an LLM to build a semantic index. For each test, the LLM generates a brief summary of the test’s purpose (e.g. “verifies user password validation”). We also assign each test a safety tier based on which modules it touches (e.g. tests covering security/authentication get Tier-1). The summaries and tier tags are converted into vector embeddings and stored in a vector database. This creates a searchable semantic catalog of the test suite.
- **Impact Analysis & Retrieval (Per Change)**: When a new commit arrives, an Impact Analyst agent uses an LLM to examine the code diff and infer the developer’s intent. For example, the diff might indicate changes to a “PrivacySettings” class. The agent summarizes the diff in natural language (e.g. “changes to user privacy settings logic”). We then perform a semantic search - the diff summary is converted into an embedding and compared against the test embeddings in the vector database. The system retrieves the top-K tests whose semantic vectors are most similar to the change intent. This yields a candidate list of tests that might be relevant based on meaning rather than just syntactic reachability.

- **Risk-Aware Ranking & Selection**: A final Selector agent ranks the retrieved tests by combining semantic relevance with test criticality. Concretely, we employ a weighted logic: a “Safety Bridge” rule ensures that any Tier-1 (mission-critical) test is selected even if its semantic match score is modest. Conversely, a “Precision Filter” discards Tier-5 (routine) tests if their semantic score is very low. This way, we guarantee safety for high-severity tests while eliminating low-value tests. The output is a prioritized, size-bounded list of tests to execute. If needed, we can iteratively expand this list by relaxing semantic thresholds.

3 Experimental Evaluation

Our evaluation of the proposed technique will help address below questions:

- (Safety) Can the LLM-based RTS select all tests that trigger known bugs? In other words, what fraction of the original failing tests is caught ?
- (Efficiency) How many fewer tests does our approach run compared to baseline RTS? We measure the percentage reduction in executed tests relative to static RTS (and Retest-All).

- (Overhead) Is the runtime overhead of LLM encoding and retrieval justified by test savings? We compare end-to-end execution time (index/query vs saved test execution).
- (Criticality Impact) How does incorporating test criticality tiers affect results? We compare recall for high-severity bugs when using the weighted ranking versus using pure semantic relevance.

3.1 Evaluation Setting

We will use the Defects4J v2.0 benchmark, which contains 835 real bugs across 17 open-source Java projects [7]. Each bug comes with a “buggy” and “fixed” version of the code and a set of fault-triggering tests that failed on the buggy version. For each bug, our agent will be given the buggy/fixed code diff (the input to Phase 2) and the preprocessed test index. The ground truth is the known set of failing tests for that bug. We measure:

- Recall (Safety) – the fraction of known failing tests that our selected subset includes;
- Selection Rate (Efficiency) – the percentage of the total test suite that was executed.

We will also record the total time spent on LLM inference and retrieval vs. the time saved by not running skipped tests. For implementation, we will use Python and LangChain, with a suitable LLM backend (e.g. OpenAI/Gemini) to perform the summarization and semantic search. We will compare three methods: Retest-All (run every test), a standard static RTS (e.g. STARTS) that selects tests by dependency reachability, and our proposed Agentic-RAG RTS. We will perform statistical analysis over all bugs to see if differences in recall and reduction are significant. For example, if our approach yields much higher reduction than static RTS at the same recall, this demonstrates that the LLM has successfully bridged the semantic gap between code changes and relevant tests.

4 Related work

- STARTS (Legunsen et al., 2017, ASE Demo) is a practical static RTS tool that builds compile-time dependency graphs to select impacted tests [1]. It significantly reduces executed tests without runtime instrumentation but may over-select compared to dynamic approaches.
- Legunsen et al. (2016, FSE) conduct a large-scale empirical study of static RTS, comparing class and method-level selection against Ekstazi [3]. They show that class-level static RTS can achieve comparable reduction but may occasionally miss regressions.
- Ekstazi (Roesler et al., 2015, ICSE Demo) is a dynamic RTS tool that tracks runtime file-level dependencies to re-execute only impacted tests [5]. It achieves substantial test reduction but introduces instrumentation overhead.
- Facebook Predictive Test Selection (Cromb et al., 2018) applies machine learning to predict failure-prone tests using historical data [2]. It achieves very high recall with substantial test reduction but depends on large-scale training data.
- Yim (2024) proposes using LLMs to select tests based on natural-language descriptions of code changes and test

metadata [6]. While similar in leveraging semantic reasoning, the approach does not incorporate a structured retrieval or ranking mechanism.

- Other Work: Prior research has explored test prioritization, reduction, and IR-based techniques for test selection [3]. Our work differs by integrating retrieval-augmented generation and explicit test criticality modeling into the RTS framework.

References

- [1] O. Legunsen, A. Shi, D. Marinov, *STARTS: Static Regression Test Selection*, 2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE), Urbana, IL, USA, 2017, pp. 949–954, doi: 10.1109/ASE.2017.8115710. Available at: <https://ieeexplore.ieee.org/document/8115710>
- [2] B. Cromb, J. Grzymiel, G. Hope *Predictive test selection: A more efficient way to ensure reliability of code changes*, DevInfra, ML Applications at Meta. Available at: <https://engineering.fb.com/2018/11/21/developer-tools>
- [3] O. Legunsen, F. Hariri, A. Shi, Y. Lu, L. Zhang, and D. Marinov, *An extensive study of static regression test selection in modern software evolution.*, In Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2016). Association for Computing Machinery, New York, NY, USA, 583–594. Available at: <https://dl.acm.org/doi/10.1145/2950290.2950361>
- [4] OpenAI articles, *Retrieval Augmented Generation (RAG) and Semantic Search for GPTs*, Available at: <https://help.openai.com/en/articles>
- [5] M. Gligoric, L. Eloussi and D. Marinov, *Ekstazi: Lightweight Test Selection*, 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering, Florence, Italy, 2015, pp. 713–716, doi: 10.1109/ICSE.2015.230. Available at: <https://ieeexplore.ieee.org/document/7203050>
- [6] Y. K. Soo, *Automatic Test Case Selection by Prompting a Large Language Model with Code Descriptions and Labeled Test Cases*, Technical Disclosure Commons, (August 22, 2024) Available at: https://www.tdcommons.org/dpubs_series/7296
- [7] R. Just, D. Jalali, and M. D. Ernst *Defects4J: a database of existing faults to enable controlled testing studies for Java programs.*, In Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA 2014). Association for Computing Machinery, New York, NY, USA, 437–440. Available at: <https://dl.acm.org/doi/10.1145/2610384.2628055>